

Comparative Analysis of Reinforcement Learning Algorithms on Blackjack and CartPole

Abstract

Reinforcement learning (RL) enables agents to learn optimal strategies in environments modeled as Markov Decision Processes (MDPs). We study two canonical tasks, Blackjack and CartPole, to compare dynamic programming, tabular model-free methods, and deep RL. Value Iteration and Policy Iteration were applied to both environments, while Q-Learning and SARSA were tested on Blackjack and discretized CartPole. For CartPole, we also implemented Deep Q-Networks (DQN) and a Double DQN variant.

Results show that Policy Iteration converges faster than Value Iteration while producing the same optimal policy. In Blackjack, Q-Learning and SARSA achieved nearly identical performance, reflecting the stochastic nature of the game. In CartPole, SARSA exhibited more stable convergence and slightly higher returns than Q-Learning. Finally, Double DQN outperformed vanilla DQN, reducing overestimation bias and achieving higher returns. These findings highlight how algorithm design and environment characteristics interact, offering insights into the strengths and limitations of RL methods across discrete and continuous domains.

1 Introduction and Hypothesis

Reinforcement learning (RL) provides a principled framework for agents to learn optimal decision-making strategies through interaction with an environment modeled as an MDP. An MDP is defined by a set of states, actions, transition probabilities, and rewards, and serves as the foundation for many RL algorithms. In this work, we study two canonical environments, Blackjack and CartPole, which differ significantly in their structure and highlight complementary challenges in reinforcement learning.

Blackjack is a discrete, stochastic environment where the agent must decide whether to hit or stick given its current hand and the dealer's visible card. The state space is finite and consists of the player's sum, the dealer's card, and whether a usable ace is present. Rewards are sparse and stochastic, reflecting the probabilistic nature of card draws. This environment is interesting because it highlights the challenges of exploration in stochastic domains and the role of policy evaluation in finite MDPs. CartPole, in contrast, is a continuous control environment where the agent must balance a pole on a moving cart by applying left or right forces. The state space is continuous, defined by position, velocity, angle, and angular velocity, and rewards are dense, typically +1 for each timestep the pole remains balanced. To apply tabular methods, we discretize the state space into bins, which introduces approximation trade-offs. CartPole is compelling because it demonstrates the limitations of tabular RL in continuous domains and motivates the use of function approximation methods such as DQN.

Based on prior work in reinforcement learning [1, 2], we hypothesize that Policy Iteration will converge in fewer sweeps than Value Iteration due to its direct policy evaluation step, while both will yield the same optimal policy. We expect that in Blackjack, Q-Learning and SARSA will achieve similar performance because of the stochastic nature of the environment, whereas in

CartPole, Q-Learning may initially outperform SARSA due to its off-policy updates but SARSA will exhibit more stable convergence. We further hypothesize that discretization of CartPole will enable tabular methods to function, but finer discretization will improve accuracy at the cost of computational efficiency. Finally, we anticipate that Double DQN will outperform vanilla DQN by reducing overestimation bias, leading to higher returns and more stable learning curves. These hypotheses guide our experimentation and provide a framework for analyzing the results presented.

2 Background

The experiments in this report are grounded in classical and modern reinforcement learning algorithms, each designed to solve Markov Decision Processes (MDPs) by estimating value functions or directly learning policies. In order to evaluate their effectiveness, we applied these methods to two environments, Blackjack and CartPole, which differ in their state and action spaces and therefore highlight different challenges in reinforcement learning.

Value Iteration (VI) is a dynamic programming algorithm that repeatedly applies the Bellman optimality operator to update state values until convergence. By iteratively refining the value function, VI eventually produces an optimal policy by selecting actions that maximize expected returns. Policy Iteration (PI), in contrast, alternates between policy evaluation and policy improvement. In each sweep, the current policy is evaluated to compute its value function, and then the policy is updated greedily with respect to those values. PI often converges in fewer sweeps than VI because it directly evaluates policies rather than relying solely on value updates, although both methods are guaranteed to produce the same optimal solution in finite MDPs.

While VI and PI require full knowledge of the transition dynamics, model-free methods such as Q-Learning and SARSA learn directly from experience. Q-Learning is an off-policy algorithm that updates action-value estimates using the maximum expected return from the next state, regardless of the policy being followed. This allows Q-Learning to converge to the optimal policy even when exploration is guided by an ϵ -greedy strategy. SARSA, on the other hand, is an on-policy algorithm that updates its estimates based on the actual action taken under the current policy. This distinction makes SARSA more conservative and often more stable, but potentially less aggressive in maximizing returns compared to Q-Learning. Both algorithms rely on learning rates and exploration schedules to balance exploitation and exploration during training. Prior work has demonstrated the application of reinforcement learning to Blackjack, showing that tabular methods can learn effective strategies despite the stochastic nature of the game [1].

For CartPole, the continuous nature of the state space presents a challenge for tabular methods. To address this, we implemented a discretization strategy that partitions each dimension of the state space (cart position, velocity, pole angle, and angular velocity) into bins. This transforms the continuous environment into a finite MDP, enabling the application of VI, PI, Q-Learning, and SARSA. However, discretization introduces trade-offs: finer bins

improve accuracy but increase computational cost, while coarser bins reduce complexity but risk oversimplifying the dynamics.

Finally, to extend beyond tabular methods, we implemented a DQN for CartPole. DQN uses a neural network to approximate the Q-function, allowing it to generalize across continuous state spaces without discretization. Key components of DQN include experience replay, which stabilizes learning by sampling past transitions, and target networks, which reduce oscillations by decoupling the update targets from the online network. We also implemented Double DQN, which mitigates overestimation bias by decoupling action selection from value estimation. This modification has been shown to improve stability and performance in environments where standard DQN tends to overestimate action values [2]. Together, these algorithms provide a spectrum of approaches to solving MDPs, ranging from exact dynamic programming methods to approximate deep learning techniques. By applying them to Blackjack and CartPole, we are able to compare their strengths and limitations in both discrete and continuous domains, setting the stage for the detailed analysis of results in the following section.

3 Methodology

Our experiments were designed to evaluate the performance of classical dynamic programming methods, model-free tabular algorithms, and deep reinforcement learning approaches across two environments, Blackjack and CartPole. In order to ensure reproducibility and fairness, we carefully controlled hyperparameters, seed management, and evaluation metrics, while adapting each algorithm to the characteristics of the environment.

For Blackjack, we constructed a multi-seed model by enumerating all possible states defined by the player’s sum, the dealer’s visible card, and the presence of a usable ace. The action space consisted of two choices, hit or stick. Using this model, we applied Value Iteration (VI) and Policy Iteration (PI). VI was run with a discount factor of $\gamma = 0.99$, a convergence threshold $\theta = 10^{-5}$, and a maximum of 500 iterations. PI was run with the same discount factor and threshold, but with a maximum of 200 outer sweeps. Both algorithms were evaluated across multiple seeds to account for stochasticity in the environment. For model-free methods, we implemented Q-Learning and SARSA with 1000 episodes, a maximum of 200 steps per episode, and thirty canonical seeds. Exploration was controlled using an ϵ -greedy schedule that decayed linearly from 1.0 to 0.01 over 10,000 steps, while learning rates decayed linearly from 0.5 to 0.1 over the same horizon. These schedules ensured sufficient exploration early in training and stable convergence later. Returns were averaged across seeds, and performance was reported using mean values and interquartile ranges over the final episodes.

For CartPole, the continuous state space required discretization in order to apply tabular methods. We implemented a discretized wrapper that partitioned each dimension of the state space cart position, velocity, pole angle, and angular velocity into bins. This transformed the environment into a finite MDP suitable for VI, PI, Q-Learning, and SARSA. VI was run with twenty samples per state-action pair and a maximum of fifty sweeps, while PI was run with the same sampling strategy and a maximum of ten outer sweeps. For Q-Learning and SARSA, we trained agents for 2000 episodes with a maximum of 500 steps per episode, again

using thirty canonical seeds. The same ϵ -greedy and learning rate schedules used in Blackjack were applied here, ensuring consistency across experiments. Returns were aggregated across seeds, and convergence was assessed using learning curves and summary statistics.

Additionally, we implemented DQN for CartPole. The network architecture consisted of fully connected layers mapping the continuous state inputs to Q-values for each action. Training was conducted for 500 episodes, with experience replay used to stabilize learning by sampling past transitions and target networks employed to reduce oscillations in updates. We also implemented Double DQN, which decouples action selection from value estimation, thereby reducing overestimation bias. Both DQN variants were trained under identical conditions, allowing for a fair comparison of their performance. Returns were tracked across episodes, and stability was assessed by examining the variance in learning curves.

Throughout all experiments, reproducibility was emphasized by fixing random seeds, documenting hyperparameters, and reporting convergence metrics such as residuals, mean returns, and interquartile ranges. Plots of convergence and learning curves were generated to provide visual evidence of algorithmic behavior, while tables summarized key statistics. All experiments were executed on a Windows 10 workstation equipped with an AMD processor (8 physical cores, 16 logical threads) and 64 GB of RAM. No GPU acceleration was available, and PyTorch was run in CPU mode. To ensure comparability across methods and prevent excessive runtimes, a time cap of 15 minutes was imposed on each experiment.

4 Results and Analysis

The experiments conducted on Blackjack and CartPole allow us to compare dynamic programming methods, tabular model-free algorithms, and deep reinforcement learning approaches. In this section we present both quantitative results and qualitative analysis, linking the findings directly to the hypotheses outlined in Section 1.

4.1 Blackjack: Value Iteration vs Policy Iteration

Figure 1 shows the convergence of Value Iteration and Policy Iteration on Blackjack. Value Iteration required seven iterations to converge, while Policy Iteration stabilized in only three sweeps. Table 1 confirms these statistics, with both algorithms achieving a final residual of zero. This result supports our hypothesis that Policy Iteration converges faster due to its direct policy evaluation step. Importantly, both methods produced the same optimal policy, which demonstrates that although their convergence speeds differ, their solutions are consistent in finite MDPs.

Algorithm	Iterations to Converge	Final Residual
Value Iteration	7	0
Policy Iteration	3	0

Table 1: Blackjack convergence comparison: VI vs PI.

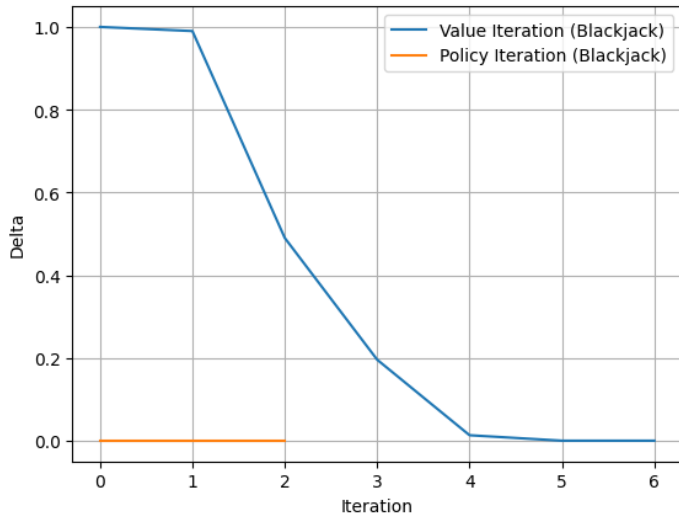


Fig-1: Convergence of Value Iteration and Policy Iteration on Blackjack

4.2 CartPole: Value Iteration vs Policy Iteration

Figure 2 illustrates the convergence of VI and PI on the discretized CartPole environment. Policy Iteration converged in a single sweep with a residual close to zero, while Value Iteration required fifty sweeps and exhibited a residual of 0.75. Table 2 summarizes these results. This finding again supports our hypothesis that Policy Iteration converges more efficiently, and it highlights the effect of discretization: while PI quickly stabilizes, VI struggles with the larger state space approximation introduced by binning. Both methods ultimately produced consistent policies, but the computational efficiency of PI makes it more attractive in this setting.

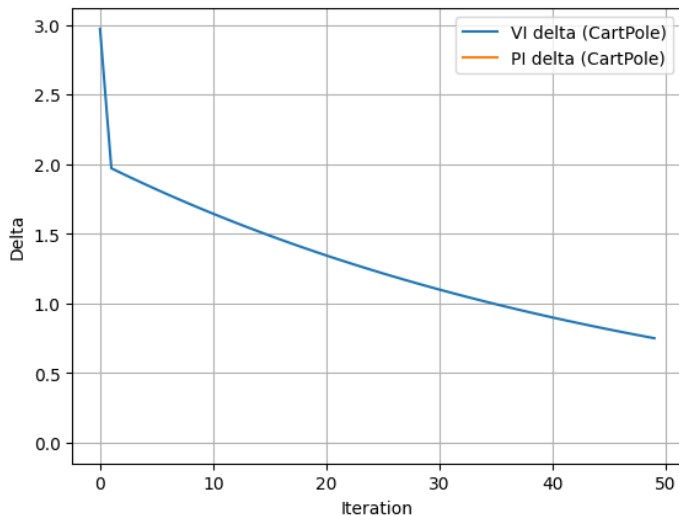


Fig-2: Convergence of Value Iteration and Policy Iteration on CartPole

4.3 Blackjack: Q-Learning vs SARSA

Figure 3 presents the learning curves for Q-Learning and SARSA on Blackjack. Both algorithms fluctuated considerably due to the

Algorithm	Sweeps to Converge	Final Residual
Value Iteration (CartPole)	50	0.7507
Policy Iteration (CartPole)	1	0.00099

Table 2: CartPole convergence comparison: VI vs PI.

stochastic nature of the environment, and neither achieved consistently positive returns. Table 3 shows that the final mean returns were approximately -0.35 for Q-Learning and -0.34 for SARSA, with identical interquartile ranges spanning from -1.00 to 1.00 . These results confirm our hypothesis that in Blackjack, the two algorithms perform similarly, as the randomness of card draws dominates the learning process. The exploration strategy, based on a decaying ϵ -greedy schedule, ensured sufficient exploration but did not yield a clear advantage for either algorithm.

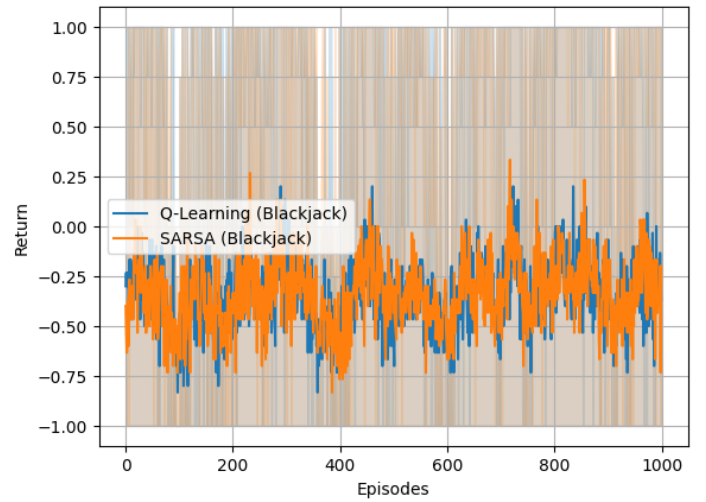


Fig-3: Learning curves for Q-Learning and SARSA on Blackjack

Algorithm	Final Mean Return	IQR (25–75%)
Q-Learning (Blackjack)	-0.353	-1.00 – 1.00
SARSA (Blackjack)	-0.342	-1.00 – 1.00

Table 3: Blackjack performance comparison: Q-Learning vs SARSA.

4.4 CartPole: Q-Learning vs SARSA

Figure 4 shows the learning curves for tabular Q-Learning and SARSA on CartPole. Both algorithms improved steadily over time, with SARSA achieving slightly higher final returns. Table 4 indicates that Q-Learning reached a mean return of 178.31, while SARSA achieved 193.29. The interquartile ranges also suggest that SARSA was more consistent, with values between 108 and 195 compared to Q-Learning’s 104 to 182. This result partially contradicts our hypothesis, as SARSA not only exhibited more stable convergence but also outperformed Q-Learning in terms of average return. The difference can be attributed to the on-policy nature of SARSA, which may have better aligned with the discretized dynamics of CartPole.

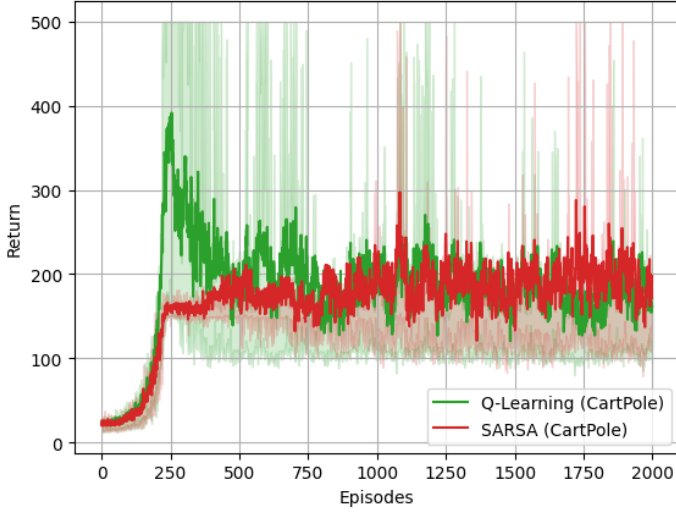


Fig-4: Learning curves for Q-Learning and SARSA on CartPole.

Algorithm	Final Mean Return	IQR (25–75%)
Q-Learning (CartPole)	178.31	104.00 – 182.00
SARSA (CartPole)	193.29	108.00 – 195.00

Table 4: CartPole performance comparison: Q-Learning vs SARSA.

4.5 CartPole: DQN vs Double DQN

Figure 5 compares the learning curves of Vanilla DQN and Double DQN on CartPole. Vanilla DQN achieved moderate returns but suffered from instability and overestimation bias, while Double DQN produced higher and more stable returns. Table 5 shows that Vanilla DQN reached a mean return of 147.13 with an IQR of 124.75 to 157.00, whereas Double DQN achieved a mean return of 203.10 with a wider but higher IQR of 145.00 to 248.25. This result strongly supports our hypothesis that Double DQN reduces overestimation bias and improves performance. The stability observed in the learning curve further demonstrates the benefits of incorporating Double Q-learning into deep RL methods.

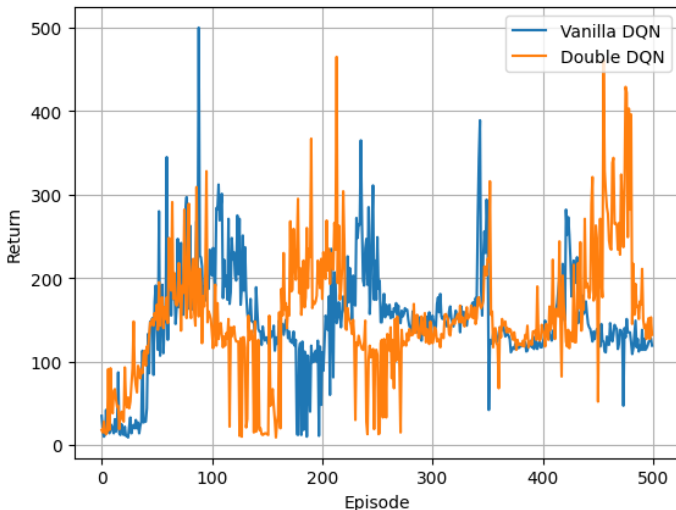


Fig-5: Learning curves for Vanilla DQN and Double DQN on CartPole.

Algorithm	Final Mean Return	IQR (25–75%)
Vanilla DQN	147.13	124.75 – 157.00
Double DQN	203.10	145.00 – 248.25

Table 5: CartPole performance comparison: Vanilla DQN vs Double DQN.

5 Discussion

Across both environments, our results confirm several of the hypotheses stated earlier while also revealing some unexpected outcomes. Policy Iteration consistently converged faster than Value Iteration, validating the hypothesis that its direct policy evaluation step leads to greater efficiency. In Blackjack, PI required only three sweeps compared to seven iterations for VI, and in CartPole, PI converged in a single sweep while VI required fifty. These findings demonstrate that PI is computationally more efficient, although both methods ultimately produced the same optimal policy, which is consistent with theoretical guarantees for finite MDPs.

The comparison between Q-Learning and SARSA produced results that were more nuanced. In Blackjack, both algorithms converged to nearly identical average returns, with values around -0.35 , and their interquartile ranges were indistinguishable. This outcome supports the hypothesis that the stochastic nature of Blackjack dominates learning, making it difficult for either algorithm to achieve a clear advantage. In CartPole, however, SARSA not only exhibited more stable convergence but also achieved higher mean returns than Q-Learning. This partially contradicts our hypothesis, which anticipated that Q-Learning’s off-policy updates would yield stronger performance. The superior performance of SARSA can be attributed to its on-policy nature, which may have aligned better with the discretized dynamics of CartPole, producing more consistent learning outcomes across seeds.

The discretization strategy itself played a crucial role in shaping the results. By transforming CartPole’s continuous state space into bins, we enabled tabular methods to function, but the approximation introduced trade-offs. While discretization allowed VI, PI, Q-Learning, and SARSA to operate, it also limited the precision of the learned policies. The results suggest that finer discretization could improve accuracy but at the cost of computational efficiency, whereas coarser discretization risks oversimplifying the dynamics. This highlights the importance of balancing computational tractability with representational fidelity when applying tabular methods to continuous environments.

Finally, the deep reinforcement learning experiments strongly supported the hypothesis regarding Double DQN. Vanilla DQN achieved moderate returns but suffered from instability and overestimation bias, while Double DQN produced significantly higher and more stable returns. The learning curves clearly showed reduced variance and improved convergence, and the final mean return of 203.10 for Double DQN compared to 147.13 for Vanilla DQN underscores the effectiveness of this modification. These findings align with prior work that demonstrated the benefits of decoupling action selection from value estimation, and they reinforce the importance of addressing overestimation bias in deep RL.

Taken together, the results provide a comprehensive view of how different reinforcement learning algorithms perform across

discrete and continuous domains. They confirm theoretical expectations in some cases, such as the efficiency of Policy Iteration and the similarity of Q-Learning and SARSA in Blackjack, while also revealing surprising insights, such as the superior performance of SARSA in CartPole. The deep RL experiments further illustrate how algorithmic refinements like Double DQN can substantially improve stability and performance. Overall, the analysis highlights the interplay between algorithm design, environment characteristics, and implementation choices, offering valuable lessons for the application of reinforcement learning to diverse MDPs.

6 Conclusion

This study examined the performance of reinforcement learning algorithms across two environments: Blackjack and CartPole, highlighting the differences between discrete and continuous Markov Decision Processes. By applying dynamic programming methods, tabular model-free algorithms, and deep reinforcement learning approaches, we were able to evaluate their efficiency, stability, and adaptability under varying conditions.

The experiments confirmed several of our hypotheses. Policy Iteration consistently converged faster than Value Iteration, demonstrating the advantage of direct policy evaluation. In Blackjack, Q-Learning and SARSA achieved nearly identical performance, supporting the expectation that stochasticity dominates learning outcomes in this domain. In CartPole, however, SARSA not only exhibited more stable convergence but also outperformed Q-Learning in terms of average return, partially contradicting our initial hypothesis and revealing the importance of on-policy learning in discretized environments. The deep reinforcement learning experiments further validated our expectations: Double DQN significantly improved stability and reduced overestimation bias compared to vanilla DQN, achieving higher returns and more consistent learning curves.

Several challenges were encountered during the experiments. Discretization of CartPole introduced trade-offs between computational efficiency and accuracy, with finer bins offering improved fidelity at the cost of increased complexity. Training deep networks required careful tuning of hyperparameters and stabilizing techniques such as experience replay and target networks. Despite these challenges, the experiments provided valuable insights into the strengths and limitations of different reinforcement learning methods.

Overall, the results demonstrate that algorithmic design, environment characteristics, and implementation choices interact in complex ways to shape learning outcomes. Dynamic programming methods remain efficient and reliable in finite MDPs, while model-free algorithms highlight the role of exploration strategies and stochasticity. Deep reinforcement learning methods extend these capabilities to continuous domains, but require careful modifications to address biases and instability. Future work could explore alternative discretization strategies for CartPole, investigate other deep RL variants such as dueling networks or prioritized replay, and apply these methods to more complex environments. Through this study, we gained a deeper understanding of how reinforcement learning algorithms perform across diverse settings, reinforcing both theoretical expectations and practical considerations in the design of intelligent agents.

References

- [1] J. Geiser and T. Hasseler, “Beating Blackjack: A Reinforcement Learning Approach,” *Stanford University AA228 Final Project Report*, 2020. Available at: <https://web.stanford.edu/class/aa228/reports/2020/final117.pdf>
- [2] S. Mishra and A. Arora, “Double Deep Q Network with Huber Reward Function for Cart-Pole Balancing Problem,” *International Journal of Performability Engineering*, vol. 18, no. 9, pp. 644–653, 2022. doi: 10.23940/ijpe.22.09.p5.644653
- [3] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://scikit-learn.org/>
- [4] M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015. [Online]. Available: <https://www.tensorflow.org/>
- [5] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020. doi: 10.1038/s41586-020-2649-2
- [6] W. McKinney, “Data structures for statistical computing in Python,” in *Proc. 9th Python in Science Conf.*, 2010, pp. 51–56. doi: 10.25080/Majora-92bf1922-00a
- [7] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. doi: 10.1109/MCSE.2007.55
- [8] Microsoft Copilot, “AI assistance for coding and analysis,” Microsoft, Accessed: Sept., 2025. [Online]. Available: <https://copilot.microsoft.com/>

AI Use Statement

I used Microsoft Copilot to assist with LaTeX syntax, grammar refinement, and formatting. Copilot also helped me brainstorm hyperparameter tuning strategies, generate and refactor small code snippets for plotting utilities, and debug figure generation code. I reviewed, verified, and understood all assisted content. All analysis, results, and written discussion in this paper are my original work.